variables except for one or two, you can use the `[=]` form of the lambda introducer but follow the equals sign with a list of variables to capture by reference preceded with ampersands. The following example will thus print `1239`, because `i` is copied into the lambda, but `j` and `k` are captured by reference:

```
int main()
{
    int i=1234,j=5678,k=9;
    std::function<int()> f=[=,&j,&k]{return i+j+k;};
    i=1;
    j=2;
    k=3;
    std::cout<<f()<<std::endl;
}
```

Alternatively, you can capture by reference by default but capture a specific subset of variables by copying. In this case you use the `[&]` form of the lambda introducer but follow the ampersand with a list of variables to capture by copy. The following example thus prints `5688` because `i` is captured by reference, but `j` and `k` are copied:

```
int main()
{
    int i=1234,j=5678,k=9;
    std::function<int()> f=[&,j,k]{return i+j+k;};
    i=1;
    j=2;
    k=3;
    std::cout<<f()<<std::endl;
}
```

If you only want to capture the named variables, then you can omit the leading `=` or `&` and just list the variables to be captured, prefixing them with an ampersand to capture by reference rather than copy. The following code will thus print `5682` because `i` and `k` are captured by reference, but `j` is copied:

```
int main()
{
    int i=1234,j=5678,k=9;
    std::function<int()> f=[&i,j,&k]{return i+j+k;};
    i=1;
    j=2;
```